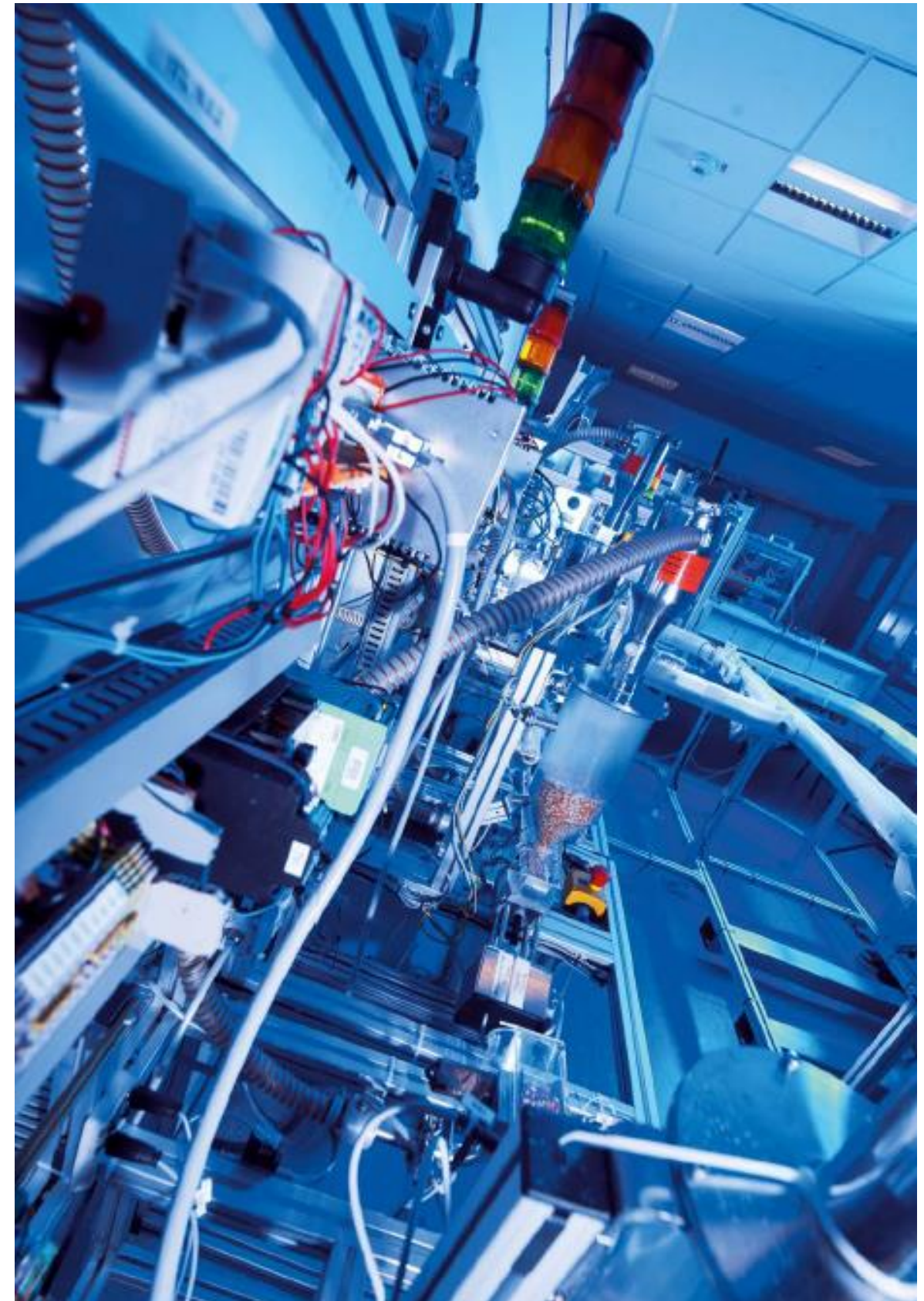


Echtzeit-Datenverarbeitung

Semaphoren

Prof. Dr. Henning Trsek

Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de



Überblick Semaphoren

- ▶ Probleme im Multitasking
 - Kooperation und Konkurrenz von Tasks
 - Deadlocks
- ▶ Semaphore und Mutex
 - Allgemeine Definition und Funktion
 - Umsetzung in freeRTOS
- ▶ Eventgroups
 - Synchronisierung/Kommunikation zwischen Tasks

Multitasking

Task 1:

Task 2:

Task 3:

steigende
Priorität

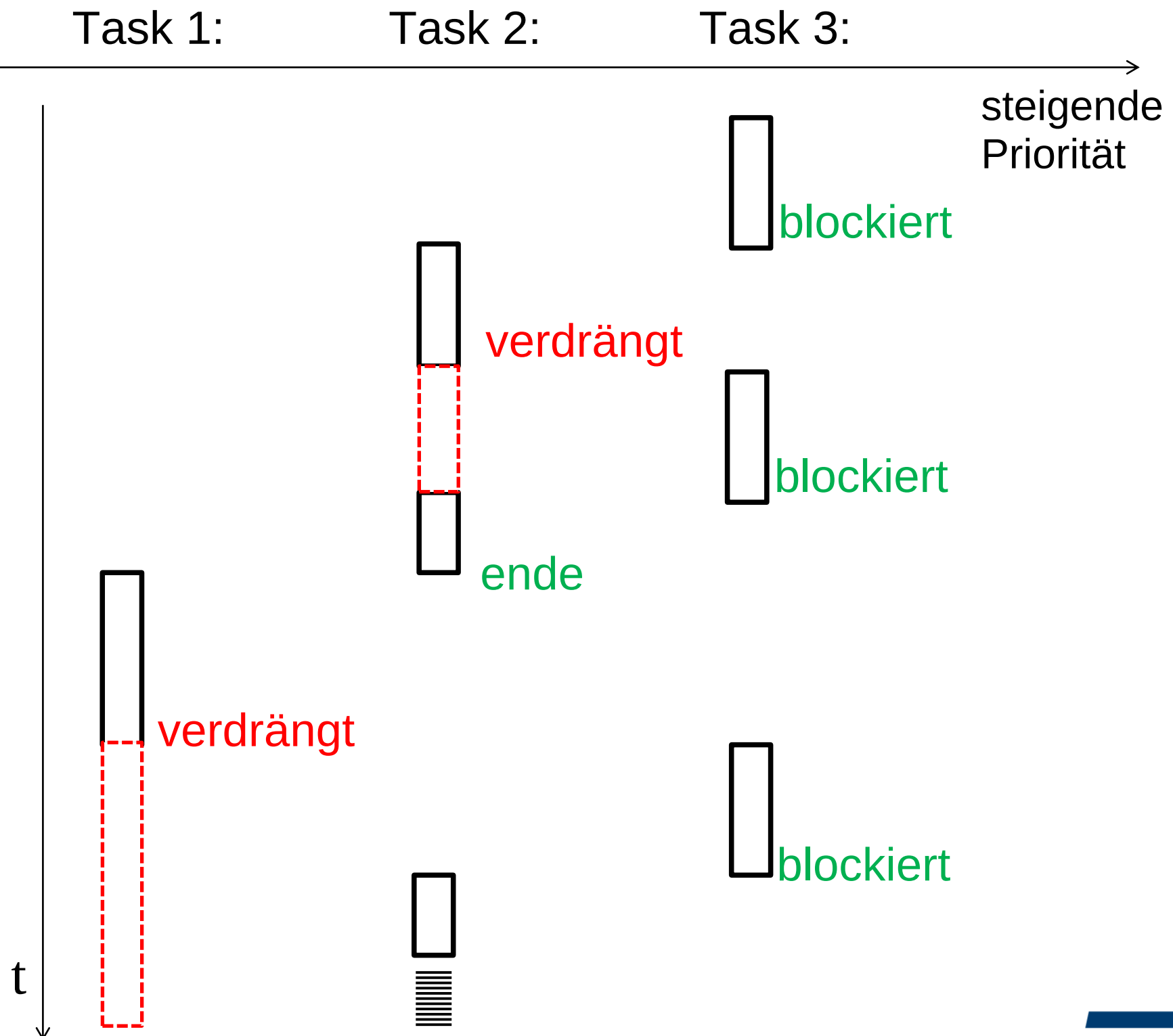


blockiert

t



Multitasking

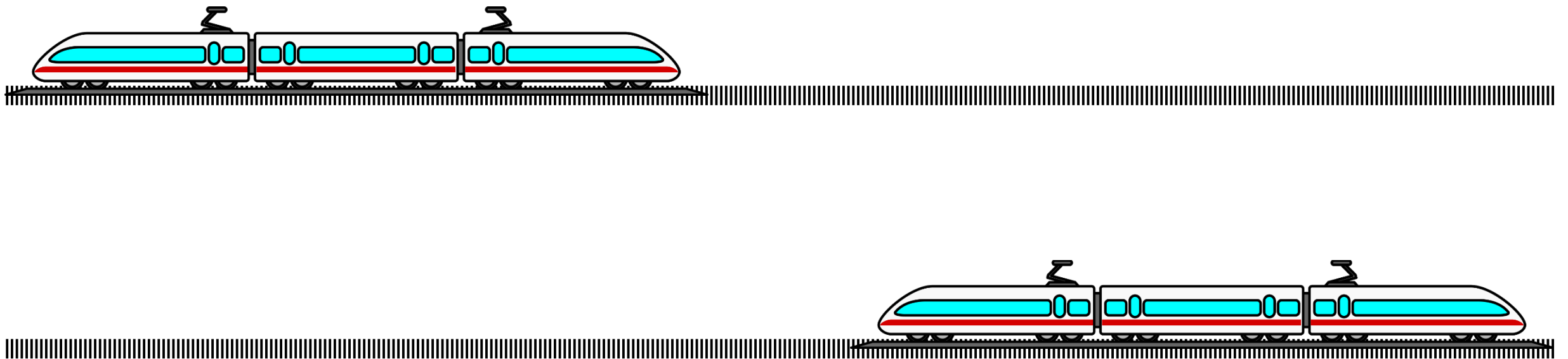


Kooperation und Konkurrenz

- ▶ **Semaphore** (Signal, Leuchtzeichen, Verkehrsampel)
Eine globale Variable, die angibt, wie viele Instanzen einer spezifischen Ressource zu Verfügung stehen.
→ „**zählende Semaphore**“
- ▶ **Mutex** (Mutual Exclusion / Gegenseitiger Ausschluss)
Eine globale Variable, die angibt, ob eine bestimmte Ressource zur Verfügung steht.
→ „**binäre Semaphore**“

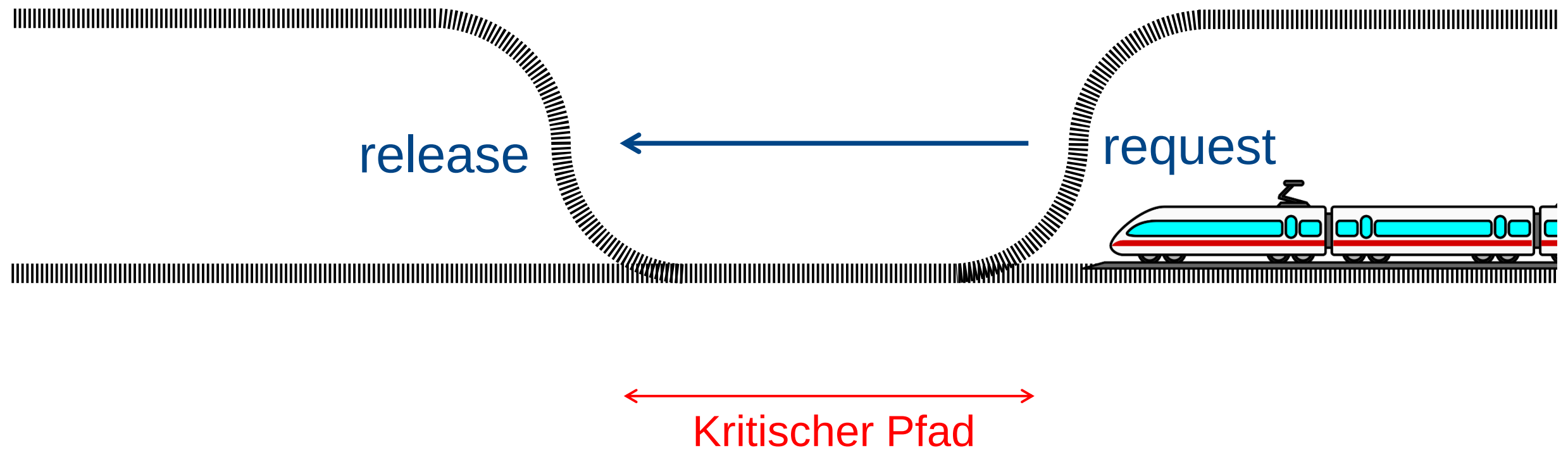
Semaphoren

Beispiel:

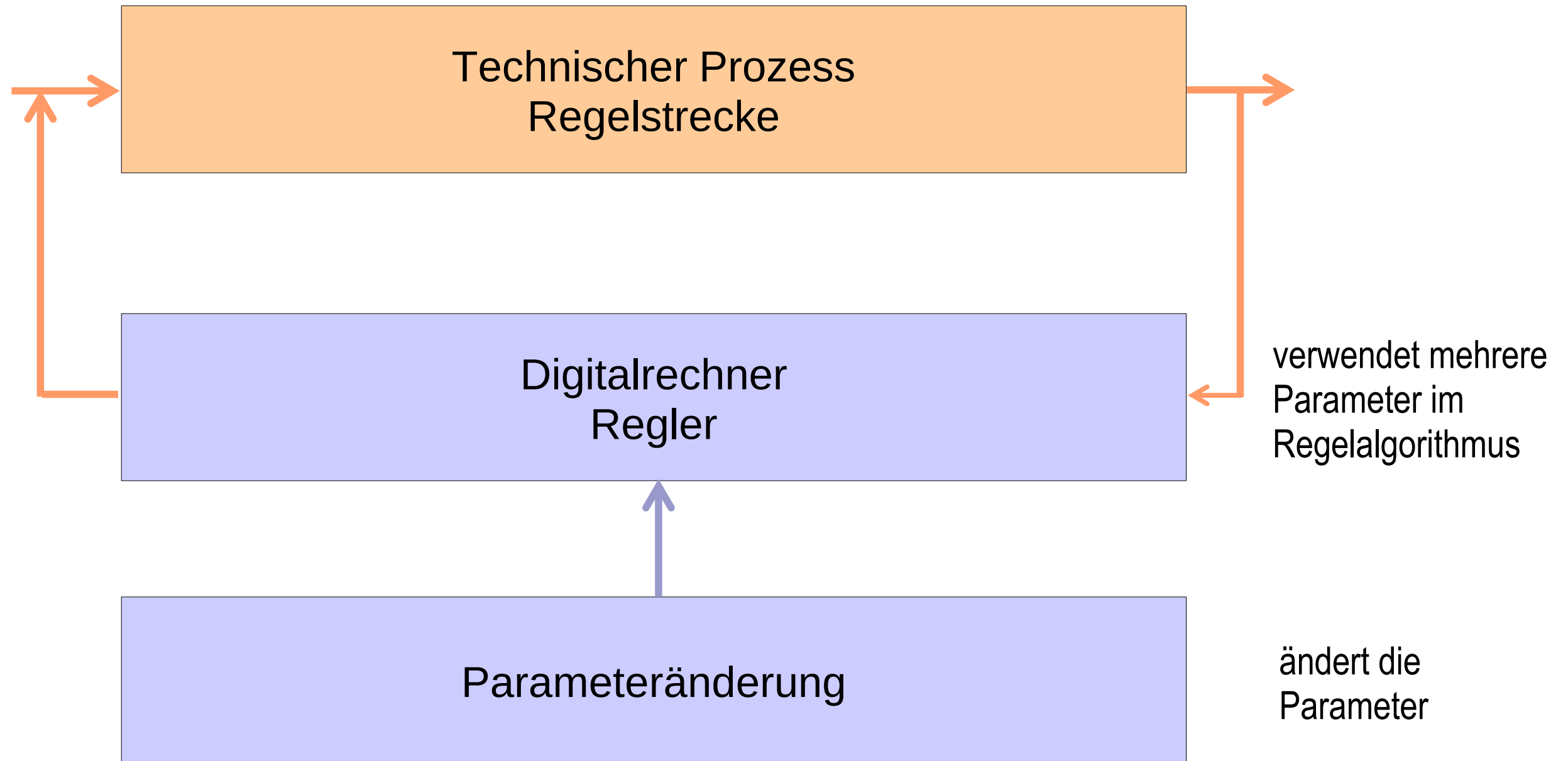


Semaphoren

Beispiel: eingleisige Bahnstrecke



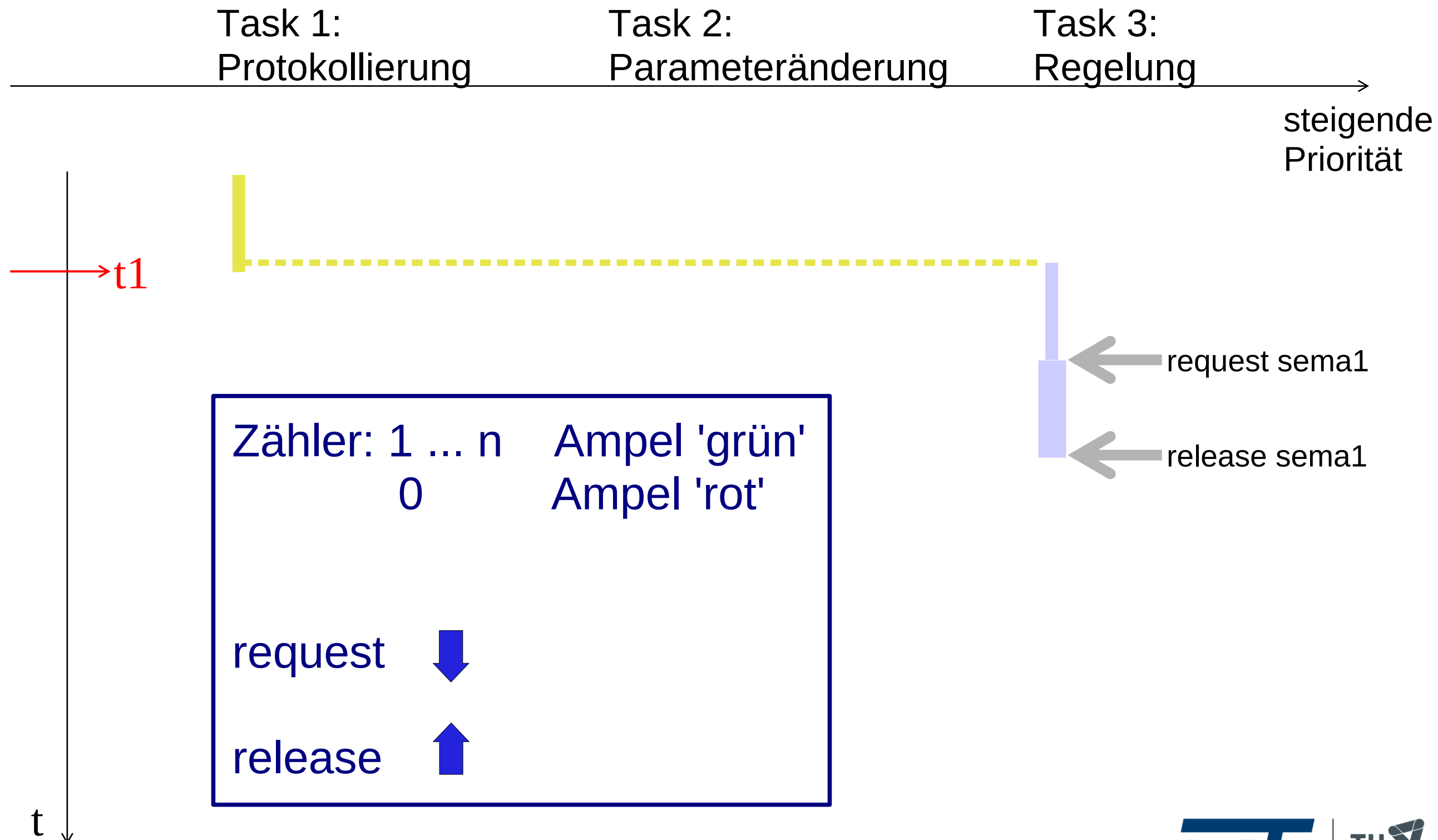
Semaphoren, Parameteränderung



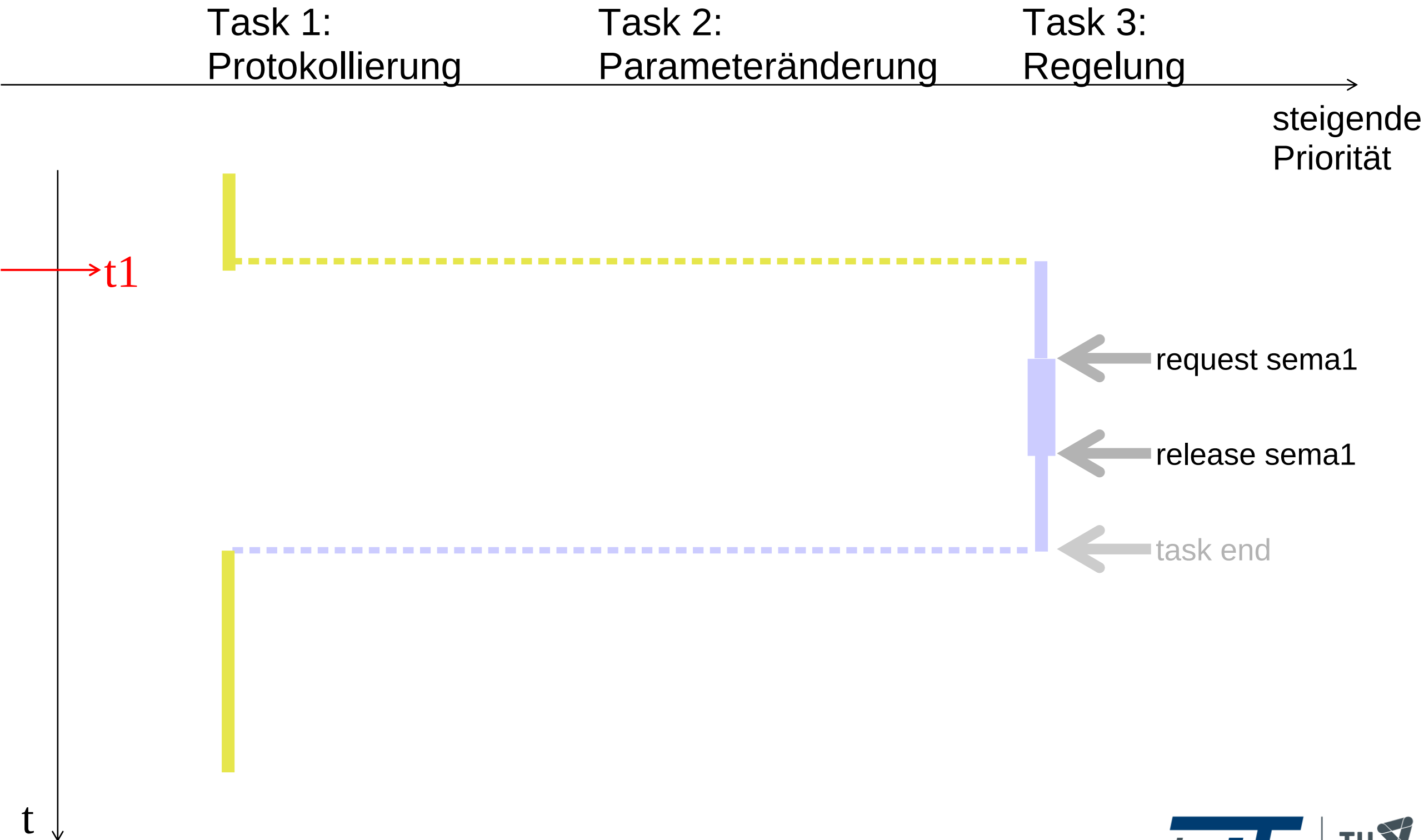
Semaphoren - zeitlicher Ablauf



Semaphoren - zeitlicher Ablauf



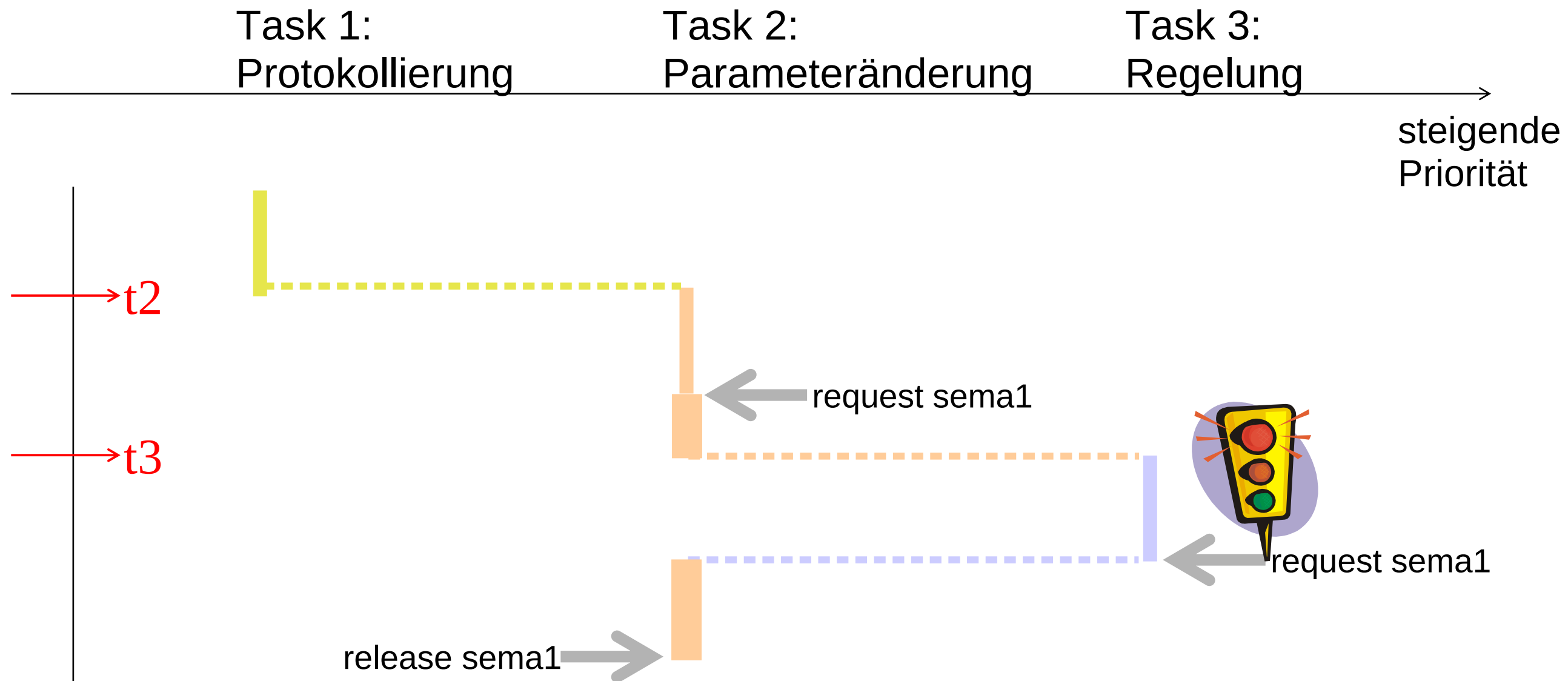
Semaphores - zeitlicher Ablauf



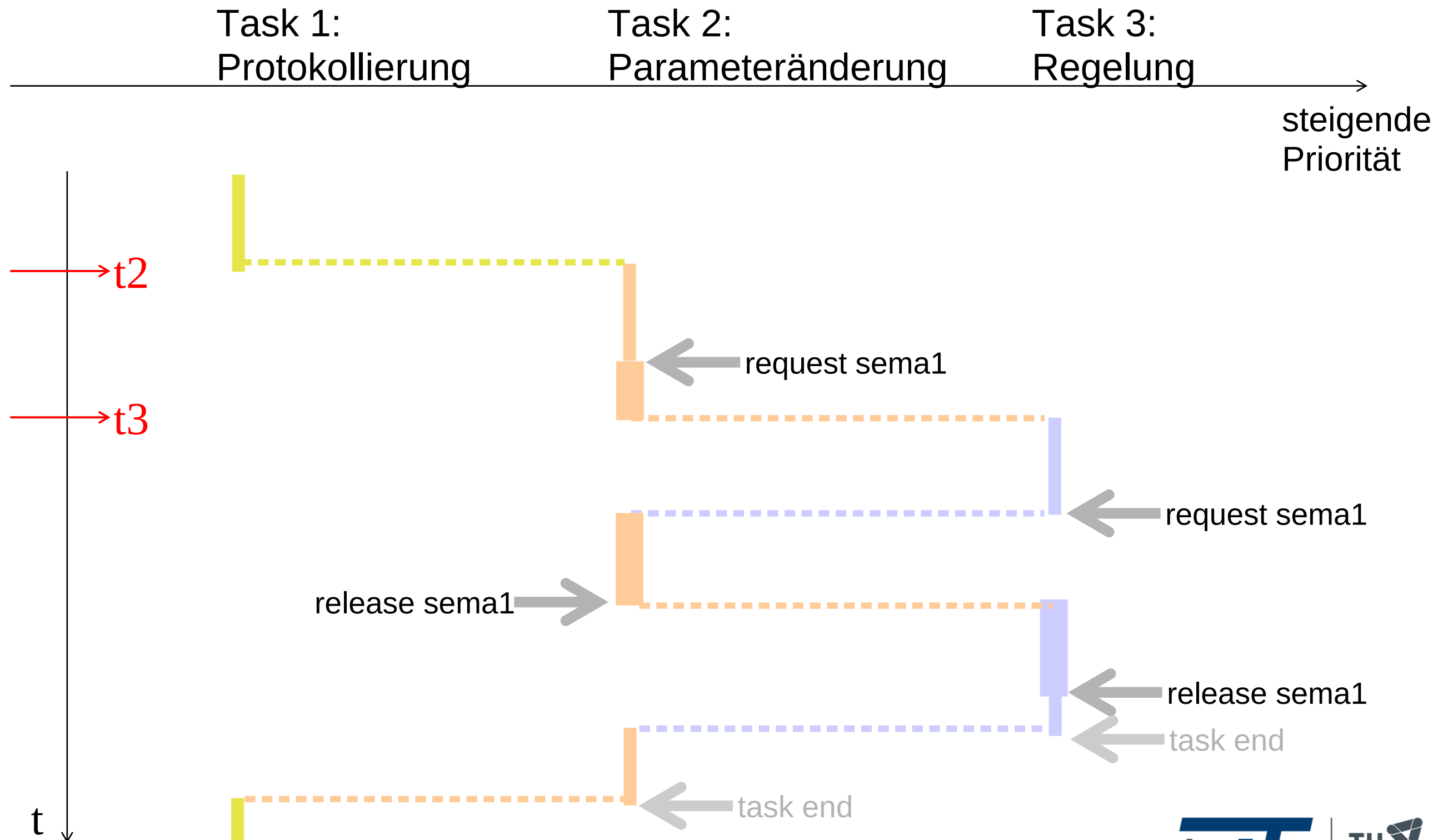
Semaphoren - zeitlicher Ablauf



Semaphoren - zeitlicher Ablauf



Semaphoren - zeitlicher Ablauf



Semaphoren

Prinzip des gegenseitigen Ausschlusses (exklusiver Zugriff).

Nichtentziehbarkeit von Betriebsmitteln.

Anzahl an Semaphoren nicht begrenzt.

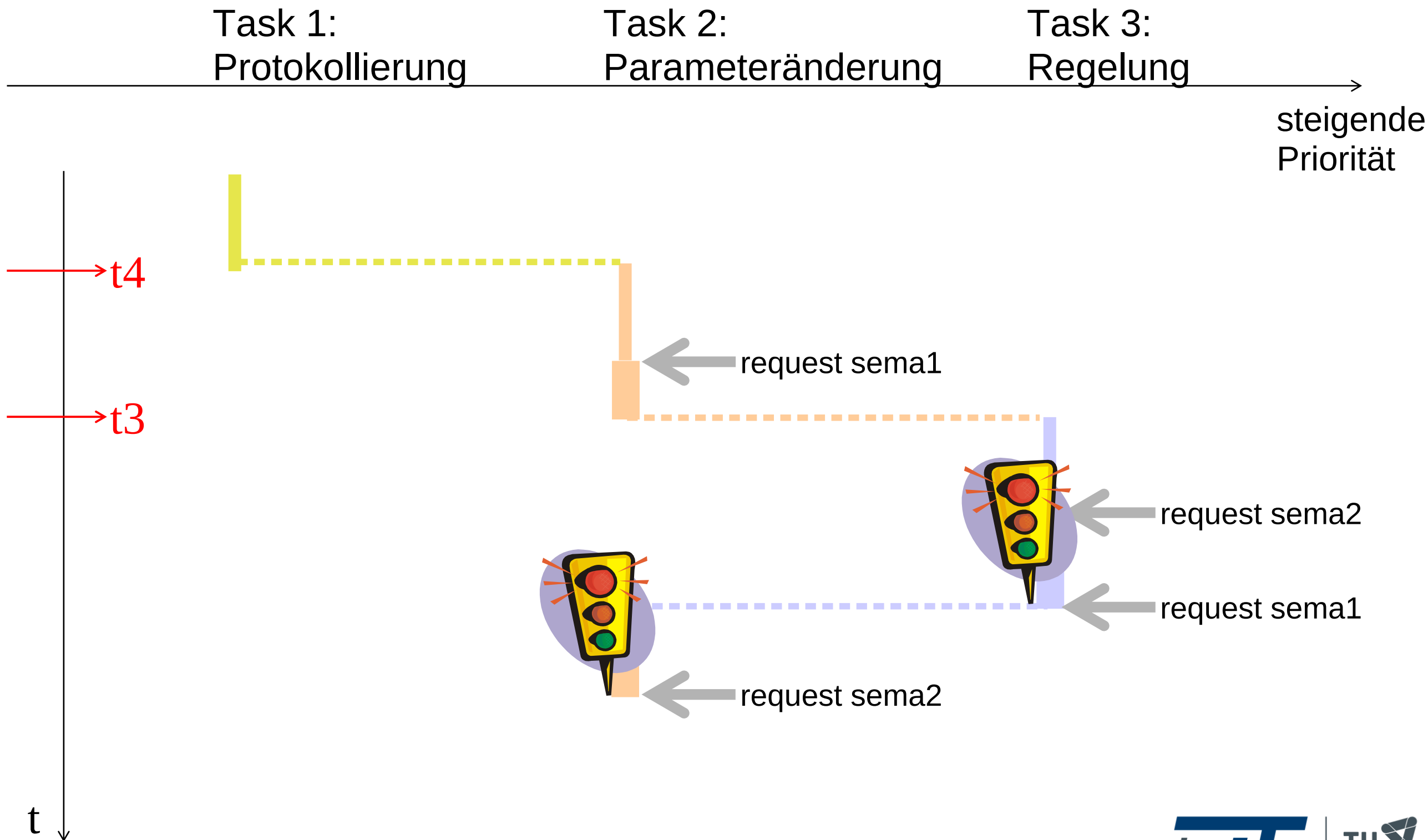
Synchronisation **nur** durch Semaphoren, **nicht** durch Prioritäten.

Semaphoren und Einplanung.

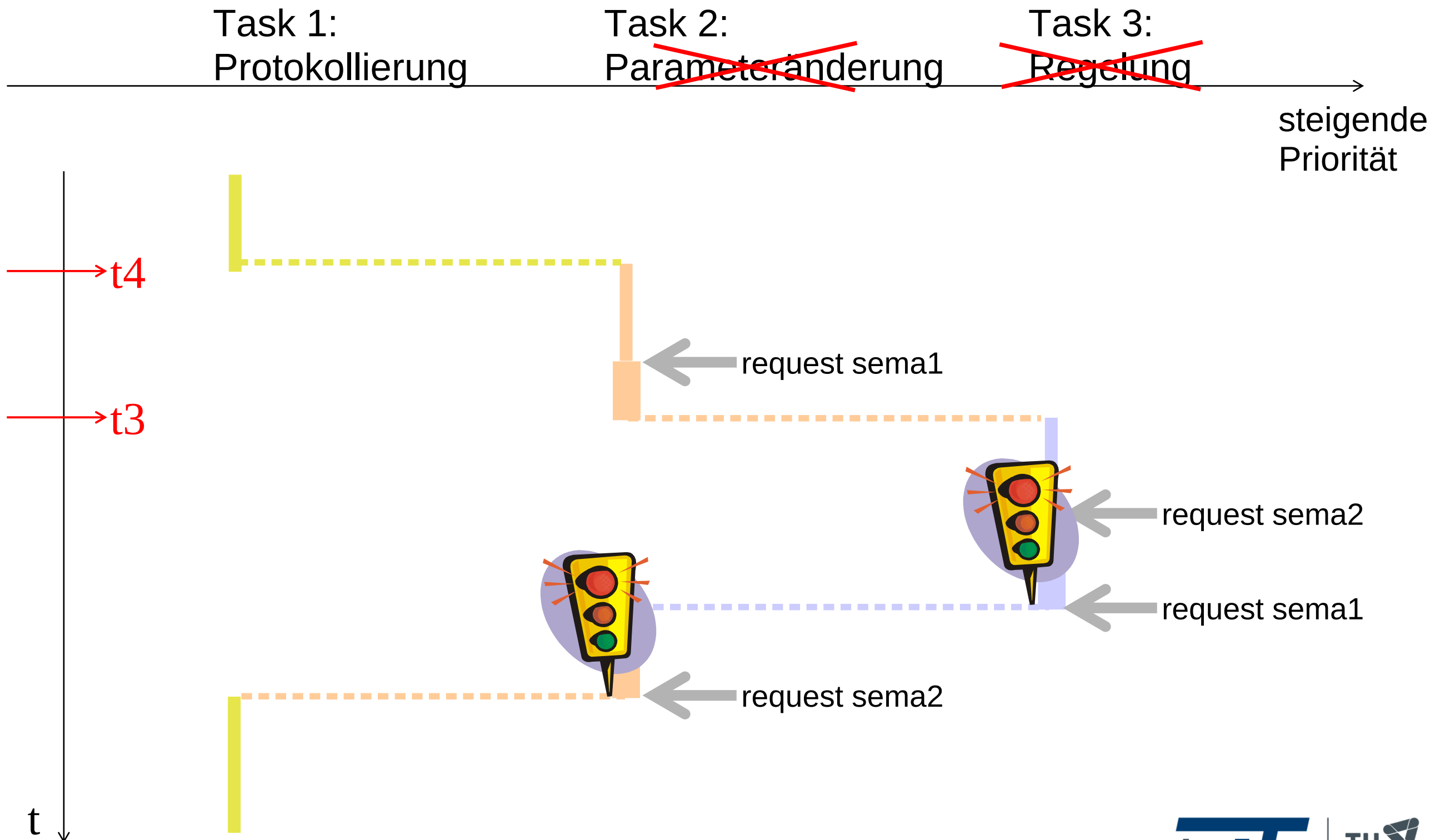
Unteilbare Operation : *Test and Set*

Vorsicht!!! Deadlock

Semaphores - Deadlock



Semaphores - Deadlock



Semaphoren in freeRTOS

1. Binäre Semaphore

Analog zum allgemeinen Mutex

```
handle = SemaphoreHandle_t xSemaphoreCreateBinary( )  
BaseType_t xSemaphoreTake(SemaphoreHandle_t handle, TickType_t ticks)  
BaseType_t xSemaphoreGive(SemaphoreHandle_t handle)
```

2. Counting Semaphore

Analog zur allgemeinen Semaphore

```
handle = SemaphoreHandle_t xSemaphoreCreateCounting( )  
BaseType_t xSemaphoreTake(SemaphoreHandle_t handle, TickType_t ticks)  
BaseType_t xSemaphoreGive(SemaphoreHandle_t handle)
```

Semaphoren sind ein Pool verfügbarer Ressourcen. Sie können auch von einer Task, die momentan keine Semaphore hält „gegeben“ werden.

In diesem Fall müssen Tasks ihre Semaphore nicht zwangsweise zurückgeben.

Mutexe in freeRTOS

In freeRTOS haben Mutexe eine leicht veränderte Funktion.

3. Mutex

```
handle = SemaphoreHandle_t xSemaphoreCreateMutex( )  
BaseType_t xSemaphoreTake(SemaphoreHandle_t handle, TickType_t ticks)  
BaseType_t xSemaphoreGive(SemaphoreHandle_t handle)
```

Im Gegensatz zu Semaphoren muss der Mutex von der Task zurückgegeben werden, die ihn gerade besitzt.

Außerdem implementiert freeRTOS eine Prioritätsvererbung für Mutexe.

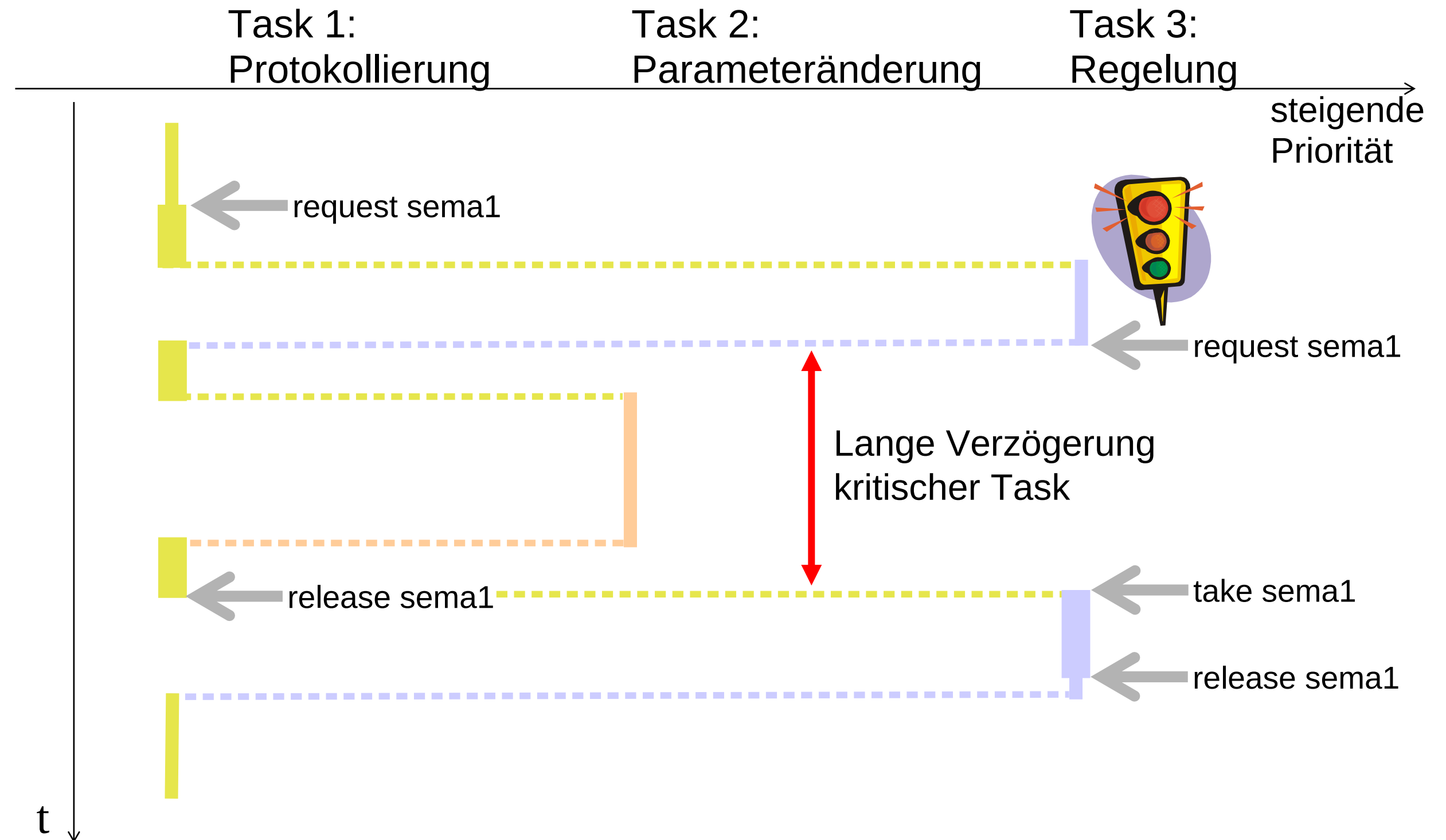
Mutexe in freeRTOS

Prioritätsvererbung bei Mutexen:

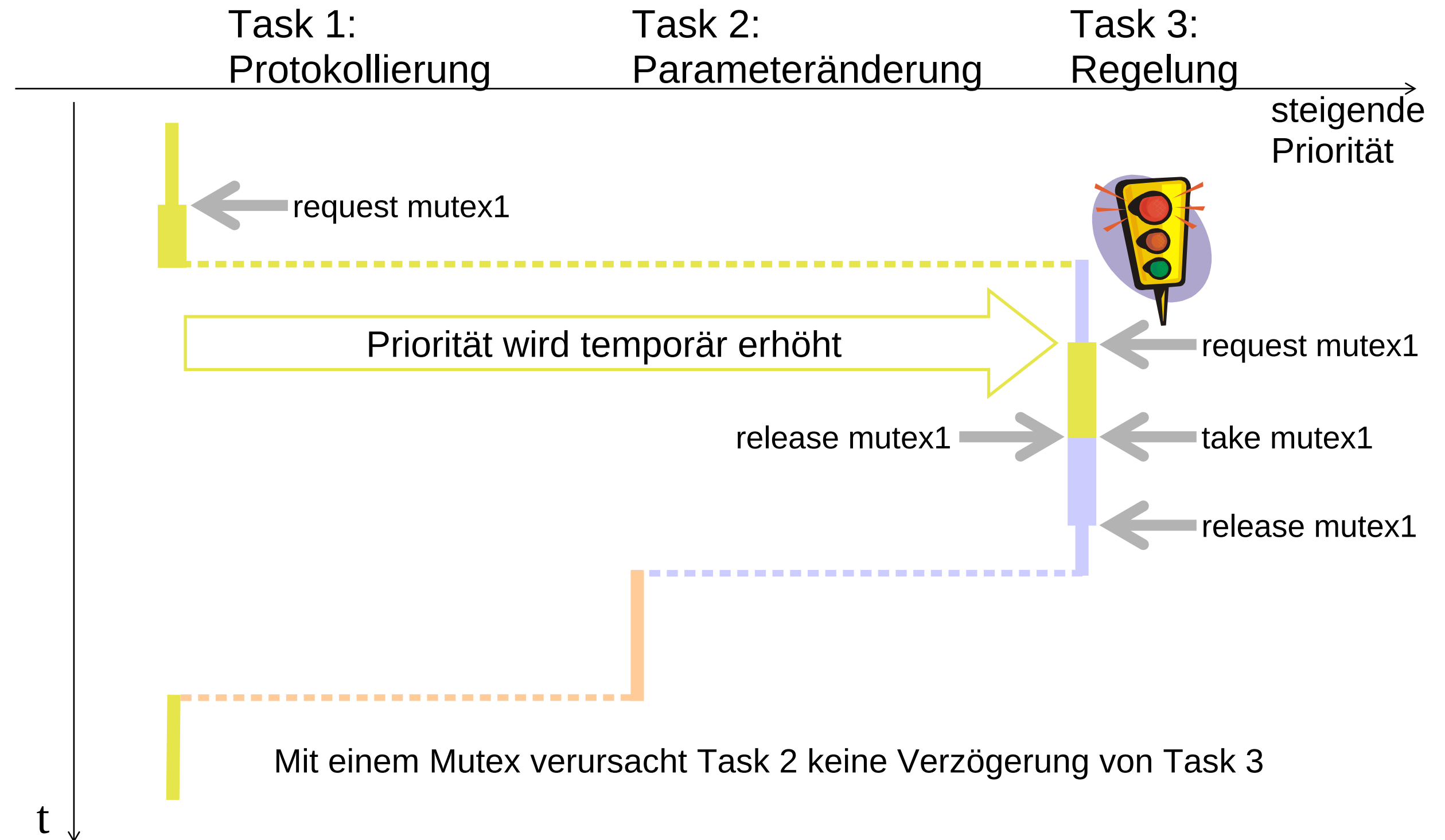
Wenn ein bereits verwendeter Mutex von einer höher priorisierten Task angefragt wird, bekommt die Task, die den Mutex momentan besitzt, temporär die Prioritätsstufe der anfragenden Task. Dadurch können Verzögerungen der wichtigeren Task minimiert werden.

Durch diese Vererbung darf ein Mutex nicht in ISRs verwendet werden.

Beispiel Mutex Prioritätsvererbung



Beispiel Mutex Prioritätsvererbung



Eventgruppen

Mit Eventgruppen stellt freeRTOS alternative Möglichkeiten zur Taskverwaltung und Synchronisierung zur Verfügung.

- Eine Eventgruppe besitzt 24 Bits, mit denen verschiedene Flags/Systemzustände dargestellt werden können.
- Tasks können auf einzelne Bits, oder Kombinationen warten.
- Tasks brauchen zur Kommunikation nicht alle Handles anderer Tasks, sondern nur den Handle einer Eventgruppe
- Bei einer Änderung in einer Eventgruppe löst das System einen Interrupt aus und prüft die Bedingungen aller Tasks, die aktuell auf ein Bit dieser Gruppe warten.
 - Es ist keine periodische Prüfung der Bedingungen oder Steuerung durch eine externe Task (wie bei `vTaskSuspend()`/`vTaskResume()`) nötig.

Eventgruppen

Erstellen einer Eventgruppe:

```
handle = EventGroupHandle_t xEventGroupCreate( );
```

Eventgruppen Bits auf 1 setzten:

```
EventBits_t xEventGroupSetBits(EventGroupHandle_t handle,  
                                EventBits_t bits);
```

Eventgruppen Bits auf 0 setzten:

```
EventBits_t xEventGroupClearBits(EventGroupHandle_t handle,  
                                  EventBits_t bits);
```

Eventgruppen Bits lesen:

```
EventBits_t xEventGroupGetBits(EventGroupHandle_t handle);
```

Eventgruppen

Die drei Funktionen Set, Clear und Get besitzen jeweils eine spezielle Version, die aus einer ISR aufgerufen werden kann.

```
BaseType_t xEventGroup___BitsFromISR(  
    handle,  
    EventBits_t bits,  
    BaseType_t *pxHigherPriorityTaskWoken );
```

Da alle FromISR Funktionen ihre Aufgaben an den zentralen freeRTOS Timer Task delegieren, wird die zusätzliche Variable *pxHigherPriorityTaskWoken benötigt.

Sie muss vor Aufruf der Funktion initialisiert werden:

```
BaseType_t xHigherPriorityTaskWoken = pdFALSE;
```

Wenn die, durch die Eventgroup geweckte, Task eine höhere Priorität hat, als die vor dem Interrupt aktive Task, wird die Variable auf pdTRUE gesetzt und ein Kontextwechsel durch den Scheduler angefragt.

Eventgruppen

Verwenden von Eventgruppen:

```
EventBits_t xEventGroupWaitBits( EventGroupHandle_t handle,  
                                  EventBits_t bits,  
                                  BaseType_t xClearOnExit,  
                                  BaseType_t xWaitForAll,  
                                  TickType_t xTicksToWait);
```

Mit `xClearOnExit` kann gewählt werden, ob die maskierten Bits nach dem erfolgreichen Lesen zurückgesetzt werden sollen.

`xWaitForAll` entscheidet, ob alle Bits in der Maske auf 1 gesetzt sein müssen, oder ob ein einzelnes Bit ausreicht. Es ist nicht möglich komplexere Checks direkt in der Funktion durchzuführen (z.B. `1&3 | 2&4`).

`xTicksToWait` gibt die Anzahl der Ticks an, nach der die Task automatisch fortgesetzt wird. Um die Task nur weiterlaufen zu lassen, wenn die korrekten Bits gesetzt sind, kann `xTicksToWait` auf `portMAX_DELAY` gesetzt werden

Beispiele

Alternative Implementierung von Aufgabe 2b mit Mutex:

```
static TaskHandle_t start_Handle = NULL;
static TaskHandle_t t1_Handle = NULL;
static TaskHandle_t t2_Handle = NULL;
static TaskHandle_t t3_Handle = NULL;
static TaskHandle_t t4_Handle = NULL;
static TaskHandle_t t5_Handle = NULL;
static SemaphoreHandle_t mutex = NULL; ←

void start_C2b(){
    printf("Aufgabe C2b\n\n");
    mutex = xSemaphoreCreateMutex(); ←
    xTaskCreate(t2, "", 128, NULL, 22, &t2_Handle);
    xTaskCreate(t1, "", 128, NULL, 21, &t1_Handle);
    xTaskCreate(t3, "", 128, NULL, 23, &t3_Handle);
    xTaskCreate(t4, "", 128, NULL, 24, &t4_Handle);
    xTaskCreate(t5, "", 128, NULL, 25, &t5_Handle);
    xTaskCreate(start, "", 128, NULL, 20, &start_Handle);

    for(;;){};
}
```

Beispiele

Alternative Implementierung von Aufgabe 2b mit Mutex:

```
static void start(){  
    xSemaphoreTake(mutex, portMAX_DELAY); ←  
    vTaskResume(t1_Handle);  
    vTaskResume(t3_Handle);  
    vTaskResume(t4_Handle);  
    vTaskResume(t2_Handle);  
    vTaskResume(t5_Handle);  
    xSemaphoreGive(mutex); ←  
    vTaskDelete(NULL);  
}
```

```
static void t1(){  
    vTaskSuspend(NULL);  
    xSemaphoreTake(mutex, portMAX_DELAY); ←  
    printf("1 ");  
    xSemaphoreGive(mutex); ←  
    vTaskDelete(NULL);  
}  
...
```


Beispiele

Alternative Implementierung von Aufgabe 2b mit Eventgruppen:

```
static TaskHandle_t start_Handle = NULL;
static TaskHandle_t t1_Handle = NULL;
static TaskHandle_t t2_Handle = NULL;
static TaskHandle_t t3_Handle = NULL;
static TaskHandle_t t4_Handle = NULL;
static TaskHandle_t t5_Handle = NULL;
static EventGroupHandle_t EventGroup; ←
```

```
void start_C2b(){
    printf("Aufgabe C2b\n\n");
    EventGroup = xEventGroupCreate(); ←
    xEventGroupClearBits(EventGroup, 1); ←
    xTaskCreate(t1, "", 128, NULL, 21, &t1_Handle);
    xTaskCreate(t2, "", 128, NULL, 22, &t2_Handle);
    xTaskCreate(t3, "", 128, NULL, 23, &t3_Handle);
    xTaskCreate(t4, "", 128, NULL, 24, &t4_Handle);
    xTaskCreate(t5, "", 128, NULL, 25, &t5_Handle);
    xTaskCreate(start, "", 128, NULL, 20, &start_Handle);

    for(;;){};
}
```

Beispiele

Alternative Implementierung von Aufgabe 2b mit Eventgruppen:

```
static void start(){  
    xEventGroupSetBits(EventGroup, 1); ←  
    vTaskDelete(NULL);  
}
```

```
static void t1(){  
    xEventGroupWaitBits(EventGroup, 1, pdFALSE, pdTRUE, portMAX_DELAY); ←  
    printf("1 ");  
    vTaskDelete(NULL);  
}
```

```
static void t2(){  
    xEventGroupWaitBits(EventGroup, 1, pdFALSE, pdTRUE, portMAX_DELAY); ←  
    printf("2 ");  
    vTaskDelete(NULL);  
}
```

...